

WAB

Provadis School of International Management and Technology

Performance of Various Python Runtimes for a Bioinformatics Algorithm

Testing the Smith-Waterman algorithm across CPython, PyPy, and
Python 3.14's native JIT

Rubin Chempananickal James
rubin.chempananickal-james@stud-provadis-hochschule.de
Matriculation Number: D876

Department: Information Technology
Module: Fortgeschrittene Programmierung
Reviewer: Prof. Dr. Jörg Daubert

24.04.2026

Abstract

Write your abstract here.

Contents

Abstract	II
List of Figures	IV
List of Tables	V
Glossary	VI
Abbreviations	VII
1. Introduction	1
1.1. Background	1
1.2. Research Questions	1
2. Methods	2
2.1. Design	2
2.2. Data Collection	2
References	i
AI Declaration	ii
Declaration of Authorship	v

List of Figures

List of Tables

Glossary

benchmark harness. The controlled code and tooling used to execute reproducible performance measurements across implementations and runtimes.

differential testing. A testing strategy that compares the outputs of multiple implementations against the same inputs to detect behavioral deviations.

local alignment. An alignment strategy that searches for the best matching subsections of two sequences instead of forcing an end-to-end alignment.

warm-up phase. The initial execution period in which a JIT-enabled runtime collects profiling information and may compile hot paths before reaching steady-state performance.

Abbreviations

CI. Continuous Integration

DP. Dynamic Programming

JIT. Just-in-Time

SW. Smith-Waterman

1. Introduction

Write your introduction here. Alright, maybe I will.

1.1. Background

1.2. Research Questions

2. Methods

Write your methods here.

2.1. Design

2.2. Data Collection

References

1. Smith TF, Waterman MS. Identification of common molecular subsequences. *Journal of Molecular Biology*. 1981;147(1):195-197. doi:10.1016/0022-2836(81)90087-5
2. Behnel S, Bradshaw R, Citro C, Dalcin L, Seljebotn DS, Smith K. Cython: The Best of Both Worlds. *Computing in Science & Engineering*. 2011;13(2):31-39. doi:10.1109/MCSE.2010.118
3. Bucher B, Ostrowski S. PEP 744 – JIT Compilation. 2024. Accessed March 10, 2026. <https://peps.python.org/pep-0744/>
4. Python Software Foundation. What's New In Python 3.14. 2025. Accessed March 10, 2026. <https://docs.python.org/3/whatsnew/3.14.html>
5. The PyPy Team. PyPy Performance. 2026. Accessed March 25, 2026. <https://www.pypy.org/performance.html>
6. Van Rossum G, Drake FL. *Python 3 Reference Manual*. CreateSpace; 2009.
7. Mariano D, Ferreira M, Sousa BL, Santos LH, Melo-Minardi RC de. A Brief History of Bioinformatics Told by Data Visualization. In: Setubal JC, Silva WM, eds. *Advances in Bioinformatics and Computational Biology*. Springer International Publishing; 2020:235-246. doi:10.1007/978-3-030-65775-8_22
8. Gorelick M, Ozsvald I. *High Performance Python*. O'Reilly Media Inc.; 2025.

AI Declaration

The usage of AI tools within this project is documented here. I declare that I have documented all interactions with AI tools, including the prompts used and the outputs received.

System	Prompt	Usage
GitHub Copilot 1	I want you to rewrite my template from LaTeX to Typst, and make it look like it did before.	Recreated the template in Typst, maintaining the original appearance.
GitHub Copilot 2	I want you to write some tests Comparing the Python 3.14 JIT Compiler, PyPy, and Cython implementations of the Smith-Waterman algorithm.	Created a test suite that benchmarks the Python 3.14 JIT Compiler, PyPy, and Cython implementations of the Smith-Waterman algorithm.
GitHub Copilot 3	Put the test code in Code/, and use Python 3.15 instead of 3.14.	Added a runtime-generated Smith-Waterman validation and benchmarking suite in Code and retargeted the project text from Python 3.14 to Python 3.15.
GitHub Copilot 4	Okay, use python 3.14. 3.15 is still in pre-release. I've created the conda env named smith and installed the packages	Retargeted the Smith-Waterman suite and Exposé references back to Python 3.14 and validated the project against the configured environment.
GitHub Copilot 5	I think the warmup runs should be timed too. We can and should somehow separate it in the final summary though.	Modified the runtime worker to time each warmup repetition individually. Added a

System	Prompt	Usage
		<code>per_run_times.csv</code> output that records every run's time with an <code>is_warmup</code> flag, keeping warmup and timed runs clearly separated in the data.
GitHub Copilot 6	It'd also be interesting to see how the performance evolves across runs per algorithm, so make a plot of that.	Added a <code>run_convergence.pdf</code> plot showing runtime relative to each case's stable median across all run indices, with the warmup region shaded, faceted by problem size.
GitHub Copilot 7	The runtime distributions per case and algorithm would also be interesting to see, maybe as a box plot?	Added a <code>runtime_distributions.pdf</code> box plot showing the distribution of runtimes per case and algorithm, faceted by problem size.
GitHub Copilot 8	Add a power law fit of the runtime scaling with input size, and plot the empirical scaling exponent.	Added an empirical scaling exponent plot (<code>scaling_exponent.pdf</code>) fitting $t \propto n^\alpha$ via OLS in log-log space with 95% CI error bars, and a warmup penalty plot. The warmup penalty plot was subsequently removed for being too confusing to interpret.
GitHub Copilot 9	Move the run convergence plotting stuff to a new jupyter notebook. I might need to isolate that to singular scenarios too. Also	Created <code>Code/plotting.ipynb</code> with interactive controls for both the run convergence and

System	Prompt	Usage
	move the scaling exponent plotting code in there.	empirical scaling exponent plots. Both sections have filter controls for scenario, problem size, and algorithm, and the scaling exponent can be faceted per scenario to compare behaviour across input types.

Declaration of Authorship

I hereby confirm that I have personally and independently prepared the present work and have not used any sources or aids other than those specified. All passages taken verbatim or in substance from other sources are identified as such. The drawings, illustrations and tables in this work are created by me or have been provided with an appropriate source reference. This work has not been submitted by me to any other university in the same or similar form for the acquisition of an academic degree.

Frankfurt, 24.04.2026

Rubin Chempananickal James